

PRUEBA TÉCNICA: INGENIERO DE AUTOMATIZACIÓN QA

Ecosistema: Plataforma de Telemetría, Monitoreo IoT y Movilidad

Cargo: Ingeniero de Automatización QA (QA Automation Engineer)

Modalidad de Entrega: Repositorio Git Público + Video de Sustentación + Documentación de Flujo

1. OBJETIVO DE LA PRUEBA

Diseñar, implementar y documentar una estrategia de pruebas automatizadas sólida, escalable y mantenible para una **aplicación móvil** y su capa de **servicios (API)**.

En AVL Mobility Solutions gestionamos miles de eventos telemáticos en tiempo real; por ello, el software de pruebas es un activo crítico de ingeniería. Evaluaremos tus capacidades en:

- **Configuración Autónoma:** Configurar e instalar frameworks de automatización móvil y backend desde cero (dependencias, variables de entorno y drivers).
- **Pensamiento Crítico:** Analizar flujos de negocio, identificar puntos de quiebre (bottlenecks) y priorizar escenarios de prueba según el riesgo e impacto al usuario.
- **Arquitectura de Pruebas:** Aplicación de buenas prácticas de diseño de código (modularidad, aislamiento de datos, reutilización y robustez ante cambios en la UI).
- **Agilidad y Co-Pilotaje:** Optimizar tus flujos de trabajo integrando asistentes de inteligencia artificial de forma transparente como aceleradores de productividad.

2. REGLAS DEL JUEGO (SIN CÓDIGO BASE)

No se proporcionará código base, bases de datos ni credenciales. Deberás resolver de forma autónoma:

1. **Aplicación Móvil (SUT):** Elige y descarga un repositorio público o ejecutable de prueba (.apk para Android o .app / .ipa para iOS).
 - Ejemplos sugeridos: [SauceLabs My Demo App](#) o [Wikipedia Mobile App](#). Usa herramientas como Appium Inspector o Maestro Studio para mapear los selectores.
2. **Capa de Servicios / API:** Utiliza una API pública de simulación o prueba (JSONPlaceholder, Restful-Booker, DummyJSON, Mockoon) para emular la autenticación, transacciones o ingesta de datos.

3. ENTREGABLES OBLIGATORIOS

Envía un enlace a una carpeta compartida en la nube (Google Drive, OneDrive, etc.) con los siguientes tres elementos:

A. Repositorio de Código Público (GitHub / GitLab)

El código debe estar estructurado de manera modular y limpia. Debe incluir:

- **Código Automatizado:** Scripts funcionales para la App Móvil y la API (y Kafka si decides tomar el reto).
- **Archivo README.md:** Requisitos previos (Node.js, Java, variables como ANDROID_HOME), comandos de instalación paso a paso y comandos para ejecutar las suites y generar reportes.

B. Bitácora de Co-Pilotaje e IA (AI_USAGE.md)

Documenta con transparencia cómo utilizaste asistentes agénticos (GitHub Copilot, Cursor, ChatGPT, Claude, etc.) para optimizar tu desarrollo:

- **Herramientas utilizadas:** Lista de asistentes y su rol en tu flujo.
- **Casos de uso específicos:** Tareas delegadas (ej. generación de boilerplate, selectores complejos, esquemas JSON).
- **Ejemplos de Prompts:** Comparte al menos dos prompts clave que aportaron un valor significativo.
- **Reflexión Técnica:** Análisis crítico de cómo influyó la IA en la velocidad de entrega y calidad del código, y cómo corregiste eventuales errores o alucinaciones.

C. Video de Sustentación Técnica

Video de **3 a 5 minutos** de duración.

- **REQUISITO EXCLUYENTE:** Debes mostrar tu rostro en cámara (Facecam) de forma visible durante la explicación. No se aceptarán grabaciones de pantalla solo con voz en off.
- **Estructura del Video:**
 - **Introducción (Máx. 45s):** Presentación personal y justificación rápida de tu estrategia de priorización.
 - **Demostración (Mín. 2 min):** Ejecución en tiempo real de las pruebas móvil y API (mostrando consola, reportes y comportamiento en emulador).
 - **Análisis Técnico (Máx. 1 min):** Explicación de problemas encontrados (sincronización, selectores) y cómo los resolviste.

4. REQUISITOS TÉCNICOS Y HERRAMIENTAS EXIGIDAS

Es obligatorio el uso exclusivo de herramientas de código abierto (Open-Source).

Capa Móvil (Foco Principal)

- **Frameworks recomendados:** Appium, Maestro CLI, Detox, o WebdriverIO Mobile.
- **Flujo obligatorio a automatizar:**
 - **Inicio de Sesión (Login):** Validar accesos exitosos y manejo de credenciales inválidas.
 - **Navegación:** Transición entre al menos dos pantallas diferentes de la aplicación.
 - **Aserciones:** Confirmación visual/lógica de un cambio de estado (ej. añadir producto, carga de un mapa o modal).
 - *Buenas prácticas:* Evita rutas XPath absolutas; prioriza localizadores estables como accessibilityId o resource-id para prevenir la inestabilidad (*flakiness*).

Capa de API (Integración y Contratos)

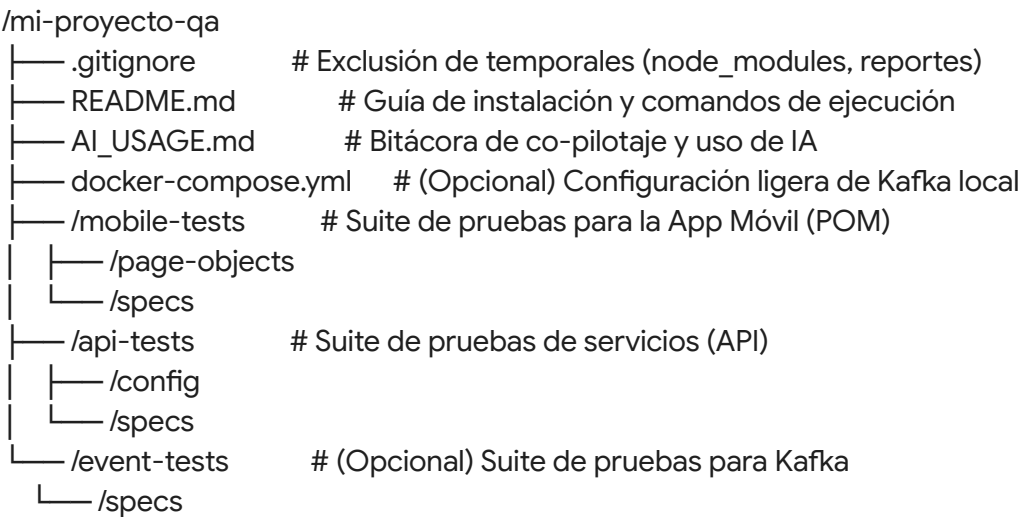
- **Frameworks recomendados:** Playwright API Testing, Bruno, Newman (Postman CLI), K6, o bibliotecas nativas (Requests/Supertest).
- **Flujo obligatorio a automatizar:**
 - **Peticiones Mutables (POST/PUT):** Envío de payloads dinámicos en formato JSON.
 - **Aserciones de Estado:** Validación de respuestas HTTP (ej. 200 OK, 201 Created) y cabeceras.
 - **Validación de Contrato (JSON Schema):** Asegurar que la estructura del payload devuelto coincida con los tipos de datos requeridos.
 - **SLA de Rendimiento:** Validar que el tiempo de respuesta sea inferior a 1.5 segundos.

Capa de Mensajería y Eventos (;Opcional y Altamente Valorado!)

Si deseas destacar, implementa una prueba automatizada para **Apache Kafka**:

- **Herramientas sugeridas:** Clientes nativos como confluent-kafka, kafka-python o kafkajs.
- **Infraestructura:** Puedes usar brokers en la nube con capa gratuita (Upstash, Confluent Cloud) o levantar un Kafka local ligero mediante docker-compose.yml.
- **Flujo a automatizar:**
 - **Productor:** Script que publique un JSON de telemetría simulado (ej: {"vehicleId": "VEH-99", "lat": 4.60, "lng": -74.08, "speed": 65}) en el tópico gps-raw-events.
 - **Consumidor:** Validar la recepción íntegra del mensaje y la aserción de su estructura de datos.

5. ESTRUCTURA RECOMENDADA DEL REPOSITORIO



6. CRITERIOS DE EVALUACIÓN

Aspecto a Evaluar	Peso	¿Qué analizaremos detalladamente?
Arquitectura Móvil	35%	Robustez, uso de esperas dinámicas explícitas, modularidad de código (POM/Screenplay) y selectores estables para mitigar la fragilidad de la prueba.
Pruebas de API y Contratos	20%	Aislamiento de variables de entorno, aserciones completas de esquemas (JSON Schema), códigos de estado y validación del SLA de respuesta.
Sustentación y Video	15%	Capacidad de síntesis en el tiempo límite (3-5 min), seguridad al argumentar la priorización de casos, cámara activa (rostro

		visible) y demo funcional.
Uso Documentado de IA	15%	Claridad en la bitácora AI_USAGE.md, calidad de prompts para destrabar problemas y juicio de ingeniería aplicado ante alucinaciones de código.
Estrategia y README	15%	Criterio técnico para priorizar según riesgos de negocio y facilidad de instalación/replicabilidad de tu entorno local mediante la documentación.
Integración con Kafka	Bonus (+10%)	Puntos adicionales directos si logras publicar y consumir de forma automatizada eventos de telemetría válidos en un tópico de Kafka.

¡Mucho éxito en este reto técnico! Esta es tu oportunidad de demostrarnos cómo lideras la calidad utilizando las mejores prácticas de automatización, agilidad con IA y criterio de arquitectura para garantizar la excelencia del producto.